

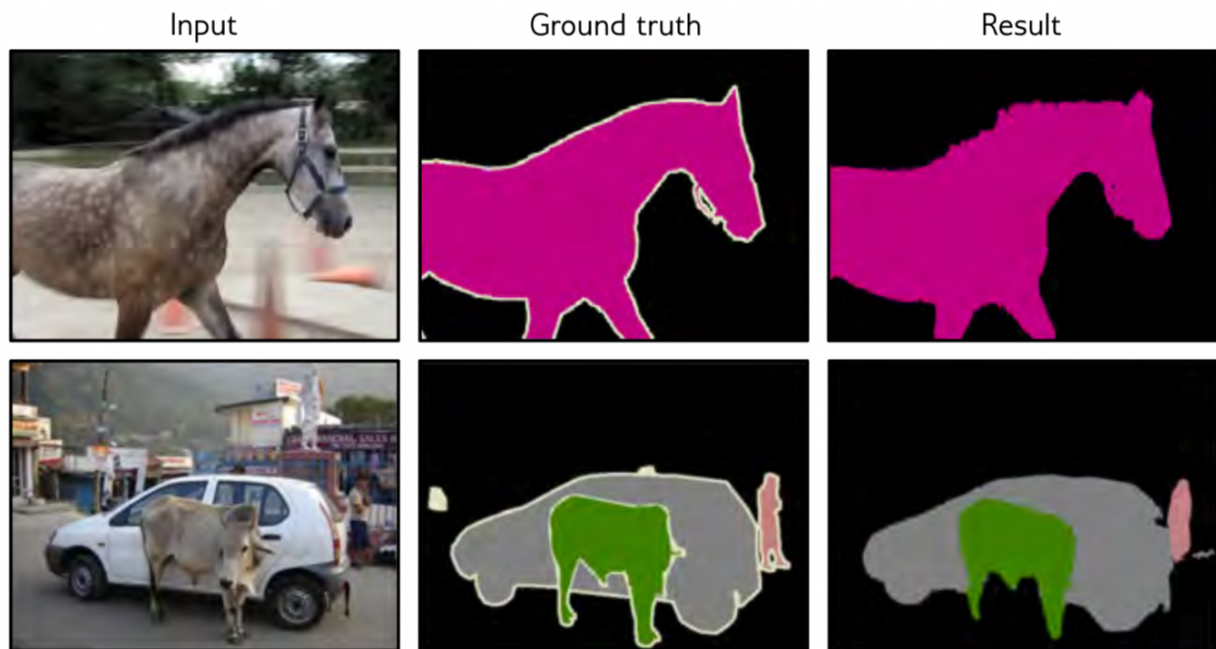


Figure 10.20 Semantic segmentation results. The final result is created from the 21 probability maps by greedily selecting the best class and using a heuristic method to find a sensible binary map based on the probabilities and their spatial proximity. If there is enough evidence, subsequent classes are added, and their segmentation maps are combined. Adapted from Noh et al. (2015).

DIMENSIONALITY REDUCTION

N.B.: This is a general approach, independent from the neural network topic.

Let's imagine a high dimension dataset. It is possible to move it to a much smaller dimension (i.e., resizing images):

$$\mathbb{R}^d \Rightarrow \mathbb{R}^m \quad (m \ll d)$$

There are some suitable techniques for it. We will see some of them, precisely:

- PCA (Principal Component Analysis); LINEAR APPROACH
- AUTOENCODERS; NON-LINEAR APPROACH

Let's start by defining PCA. The idea is to find a linear transformation in order to reduce the input dimension.

PCA

$$S = \{x_1, \dots, x_m\}, \quad x \in \mathbb{R}^m$$

We bring a matrix $W \in \mathbb{R}^{m \times d}$ s.t.:

$$x \mapsto Wx, \quad Wx \in \mathbb{R}^d \quad \text{N.B.: } x \text{ is the column vector } \vec{x}$$

To be more precise:

$$m \times \frac{d}{m} \quad \begin{matrix} | \\ x \\ | \end{matrix} \quad d \Rightarrow \left. \begin{matrix} | \\ x' \\ | \end{matrix} \right\} m$$

In general, Wx can be seen as the COMPRESSED VERSION OF x .
In opposite, it could be also necessary to UNCOMPRESS the input.
It is possible to find another transformation that extracts from Wx the original input:

$$y = Wx \quad \text{COMPRESSED INPUT}$$

$$\hat{x} = Uy = UWx \in \mathbb{R}^d, \quad U \in \mathbb{R}^{d, m}$$

Actually, this kind of compression has no quality restriction. In our case, we want to guarantee the minimum error by similarity between the input, the compressed input and the uncompressed input.

$$\underbrace{x \mapsto Wx \mapsto UWx = \hat{x}}_{\substack{| \\ \text{As similar as} \\ \text{possible}}}$$

let's see an example:

$$\left. \begin{array}{l} \langle 2, 4 \rangle \\ \langle 3, 6 \rangle \\ \langle 4, 8 \rangle \\ \langle 8, 16 \rangle \\ \langle 20, 40 \rangle \end{array} \right\} x \in \mathbb{R}^2 \rightarrow Wx \in \mathbb{R}^1 \rightarrow \hat{x} = UWx = \begin{array}{l} \langle 2 \rangle \\ \langle 3 \rangle \\ \langle 4 \rangle \\ \langle 8 \rangle \\ \langle 20 \rangle \end{array}$$

Of course, x is easily recoverable because there is a strong correlation between the 2 columns of x .

This means that, as soon as some components depend to others, and in a high dimension input it is very possible both linearly or non linearly, the similarity between the inputs, compressed or uncompressed, could be very near considering a proper reduction vector.

To minimize the reduction error, it is possible to compute the RECONSTRUCTION ERROR:

$$\sum_{i=1}^n |x_i - \overbrace{UWx_i}^{\hat{x}_i}|^2 : \text{RECONSTRUCTION ERROR}$$

and then minimize it by finding:

$$(1) \quad \underset{U \in \mathbb{R}^{d,m}, W \in \mathbb{R}^{n,d}}{\text{argmin}} \sum_{i=1}^n |x_i - UWx_i|^2$$

W : COMPRESSION MATRIX

U : UNCOMPRESSION MATRIX

It is provable that U and W are strictly related s.t. it

is not necessary to find both.

LEMMA

Let (U, W) be a solution to (1), then:

- the columns of U are orthonormal ($U^T U = I$ of $\mathbb{R}^m$)
- $W = U^T$

then, the problem in can be equivalently written as follows:

$$(2) \operatorname{argmin}_{U \in \mathbb{R}^{d, m}} \sum_{x_i \in S} |x_i - UU^T x_i|^2$$

THEOREM

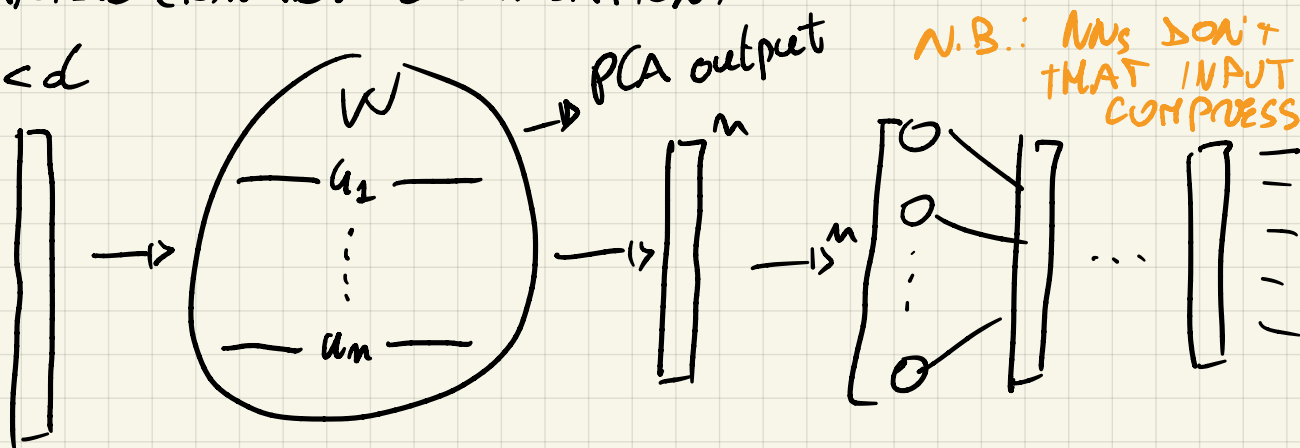
Let $S = \{x_1, \dots, x_m\} \subseteq \mathbb{R}^d$. Let $A = \sum_{i=1}^m x_i x_i^T$, and let $u_1, \dots, u_m$ be the eigenvectors of A corresponding to the largest eigenvalues of A . Then, the solution to (2) and (1) by letting $W = U^T$ is the matrix U whose columns are $u_1, \dots, u_m$.

$$U = \begin{bmatrix} | & & | \\ u_1 & \dots & u_m \\ | & & | \end{bmatrix} \rightarrow W = U^T$$

$$x \rightarrow U^T x \quad] \quad n < d$$

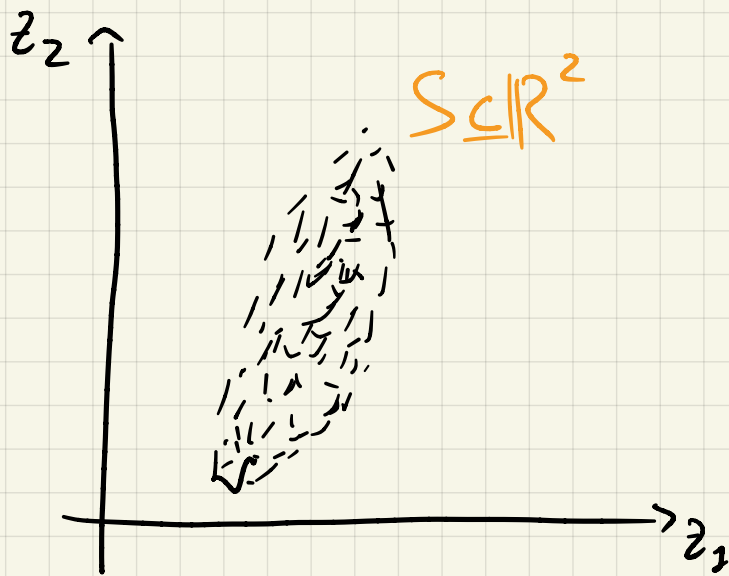
EXAMPLE (BINARY CLASSIFICATION)

• $m < d$



The purpose of dimensionality reduction is to represent the data in an optimized version, taking note of the reconstruction error.

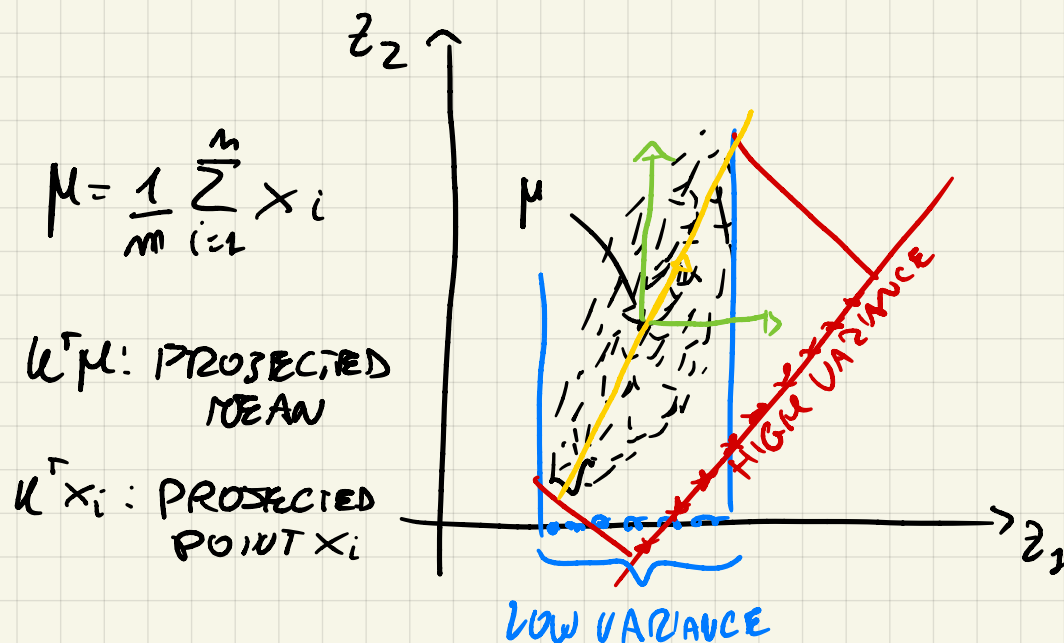
Also, reconstruction error minimisation can be interpreted as variance maximization:



Let's say I want to reduce S dimensionality:
 $S \subseteq \mathbb{R}^2 \rightarrow \mathbb{R}^1$

This projection will have some variance, as some points could collapse on each other onto their projection.

So, by searching for a projection that maximizes variance, I reduce the probability of having collapsing points.



Let's say I am projecting my points onto the yellow line:

VARIANCE OF PROJECTED POINTS

$$\sigma_s^2 = \frac{1}{m} \sum_{i=1}^m [u^T x_i - u^T \mu]^2 = u^T S u$$

$$S = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)(x_i - \mu)^T = \frac{1}{m} X^T X, \text{ for } X = \begin{bmatrix} (x_1 - \mu)^T \\ \vdots \\ (x_m - \mu)^T \end{bmatrix}$$

By applying normalisation (the matrix x in which every point is subtracted by μ), all the points are centered on $\mu=0$.

Now, I can find the maximum variance:

$$\max_{u \in \mathbb{R}^d} u^T S u \quad \text{where } u^T u = 1$$

In general ($x_i \in \mathbb{R}^d$, $u \in \mathbb{R}^d$):

$$\max_{u \in \mathbb{R}^d} u^T S u \quad \text{where } u^T u = 1 \in \mathbb{R}^d$$

$$S = \frac{1}{n} x^T x, \quad x \in \mathbb{R}^d$$

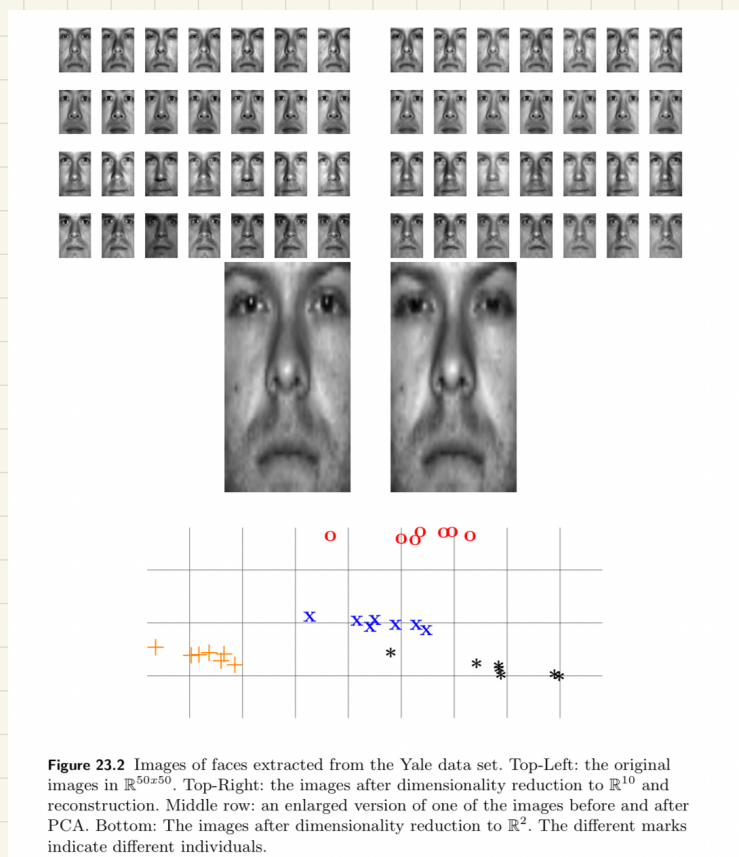
I could also say that:

$$S u = \lambda u$$

s.t. u is an eigenvector of S .

So, if u is eigenvector of S with highest λ (eigenvalue), then variance is maximal.

EXAMPLE

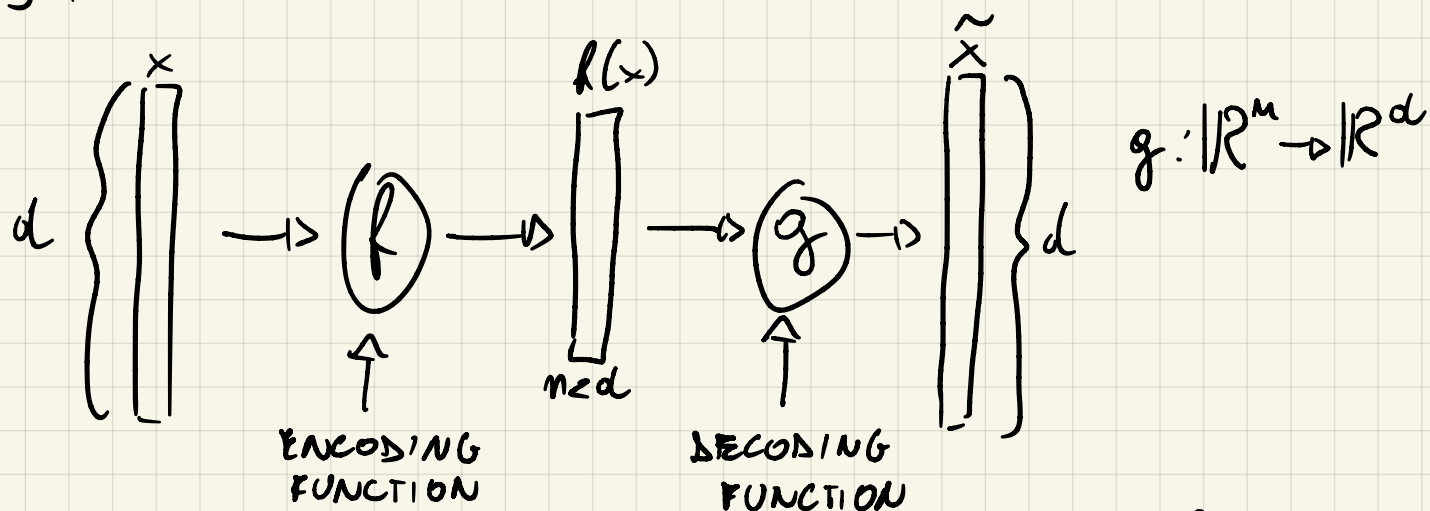


AUTOENCODERS (NON-LINEAR)

$$x \rightarrow f(x)$$

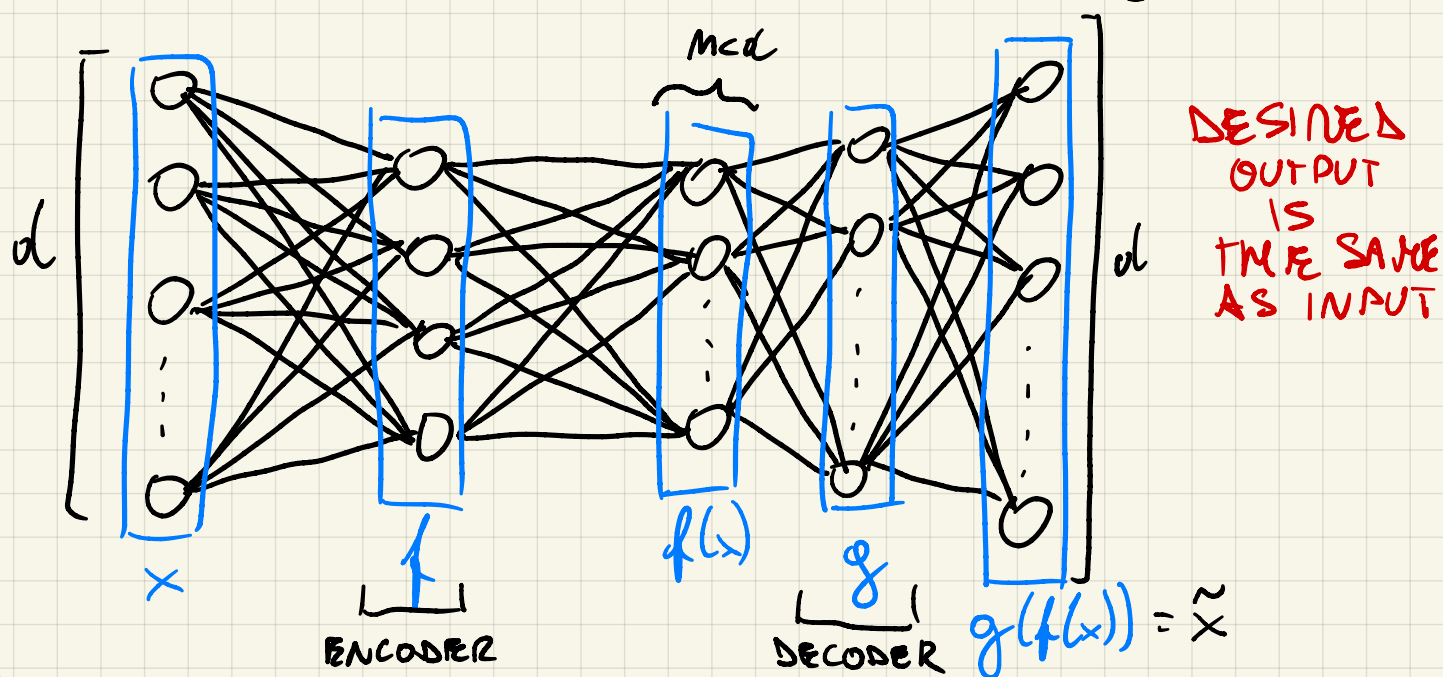
$$x \in \mathbb{R}^d, f: \mathbb{R}^d \rightarrow \mathbb{R}^m, m < d$$

In this case, we want to apply reduction of dimensionality, but in a non-linear case:



RECONSTRUCTION ERROR:
$$L(x) = \sum_{x \in S} |x - g(f(x))|^2$$

Now, the problem is to find $f(x)$ and $g(x)$. To solve this, the most useful model is neural networks. So we build a neural network in order to find $f(x)$ and $g(x)$.

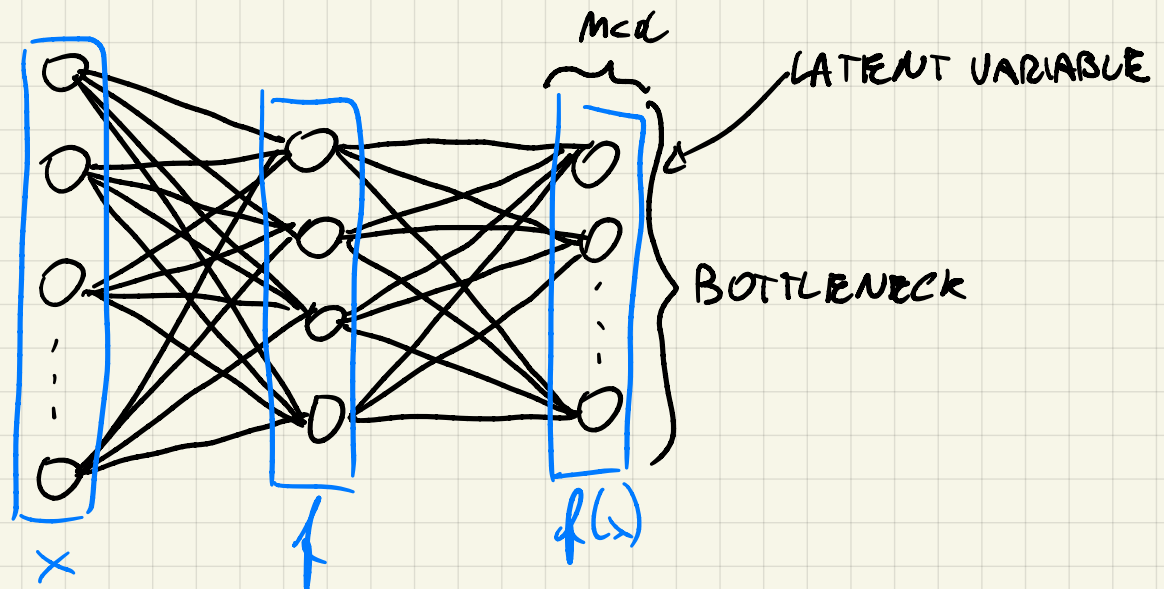


So, we train this nn with the loss function:

$$L(x) = \sum_{i=1}^m |x_i - g(f(x_i))|^2 \quad \text{RECONSTRUCTION ERROR}$$

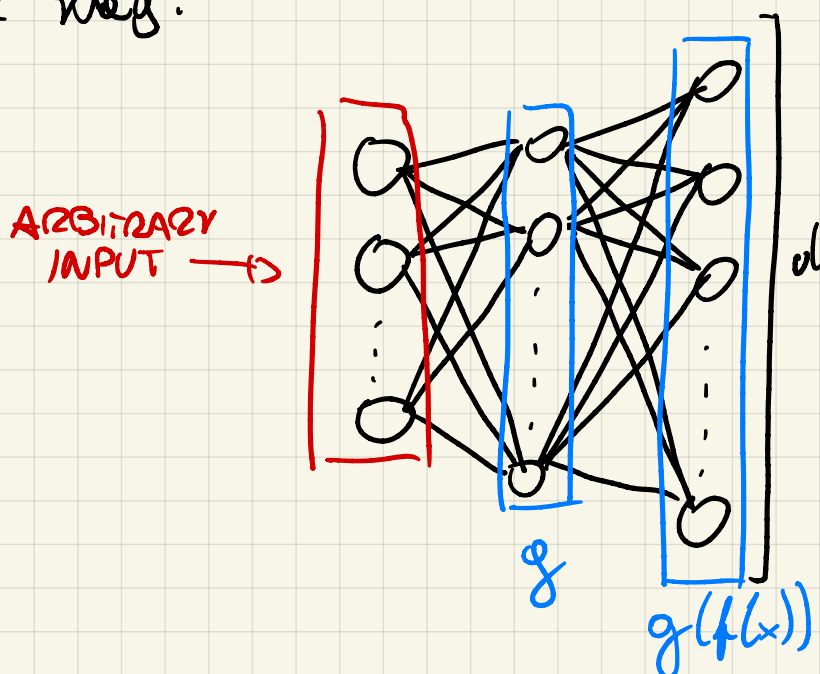
If our goal is to obtain just the reduced version of x , $f(x)$:

- Train the full nn
- Remove g and $g(f(x))$ layers



This is possible because the original nn is fully dependent on how f works (if some data is lost during encoding, g can't recover it). So the cut version of the nn is sufficient enough to compute dimensionality reduction.

It is important to say that $f(x)$ must be a BOTTLENECK, where dimensionality of x is actually reduced. We will often see bottlenecked encoders. But what about the decoder? Can it be used in some way?



Actually, it is possible to give the decoder a random input and generate an output similar to the input. However, it is necessary to continue training the NN in order to make it work. This is a generic example of generative AI.

COVERED IN CH. 14.1 OF

DEEP LEARNING BY GOODFELLOW, BENGIO, COURVILLE

N.B.: if linear functions are applied to f and g , they are basically equal to U and W . So the result will be a convex function where it is possible to find the local minimum or, in some cases, output could be the input itself. || (however, it is way more complicated than US P2A)

ENSEMBLE METHODS

This topic is about combining some methods previously introduced to implement a more powerful approach to learning.

To start with, let's formalize the idea of WEAK LEARNER

γ -WEAK LEARNABILITY

A learning algorithm A is called a γ -WEAK LEARNER for a class if there exists a function $m_H: (0, 1) \rightarrow \mathbb{N}$ s.t. for every $\delta \in (0, 1)$, for every distribution D over X , and for every labeling function $f: X \rightarrow \{-1, +1\}$ if the reliability hypothesis holds (with D, H, f) then when running A on $m \geq m_H(\delta)$ i.i.d. examples generated by D and labelled by f , A returns an hypothesis h s.t. $L(D, f) \leq \frac{1}{2} - \gamma$, with probability $p \geq \delta$ || it says that it is not good predictor but it is better than a random guess